

Clase 06 - funciones SQL y agregaciones

Sesión 1. Operadores lógicos y filtros

0. Usuarios, esquemas y permisos de prácticas

0.1 Usuario y esquema en Oracle

En Oracle, el **usuario** y el **esquema** están directamente relacionados. Puedes crear un usuario nuevo para cada práctica o bien trabajar siempre desde el mismo usuario utilizando prefijos para diferenciar los casos de uso, por ejemplo:

Caso práctico	Prefijo recomendado	Ejemplo de tablas
Veterinaria	VET_	VET_clientes , VET_mascotas ...
Academia	ACA_	
Biblioteca	BIB_	
Tienda	TIE_	TIE_clientes , TIE_compras
Empresa	EMP_	EMP_clientes , ...

0.2 Crear un usuario de prácticas "amplio", pero controlado

```
-- Utiliza el nombre que tu quieras
CREATE USER alumno01 IDENTIFIED BY Alumno01_123
DEFAULT TABLESPACE users
TEMPORARY TABLESPACE temp
QUOTA UNLIMITED ON users;
```

Permisos recomendados:

```
GRANT CREATE SESSION TO alumno01;
GRANT CREATE TABLE TO alumno01;
GRANT CREATE VIEW TO alumno01;
GRANT CREATE SEQUENCE TO alumno01;
GRANT CREATE PROCEDURE TO alumno01;
GRANT CREATE TRIGGER TO alumno01;
GRANT CREATE SYNONYM TO alumno01;
GRANT CREATE TYPE TO alumno01;
GRANT SELECT_CATALOG_ROLE TO alumno01;
```

0.3. Crear Conexión en SQL Developer

Una vez creado el usuario, debes crear su conexión en SQL Developer.

Datos orientativos:

```
Nombre de conexión: el-que-tu-quieras
Usuario: alumno01
Contraseña: Alumno01_123
Host: localhost
Puerto: 1521
Servicio: FREEPDB1
```

Después debe pulsar en **Probar** y luego **Conectar** .

0.4 Script de preparación

Este bloque solo debe ejecutarse si no tienes las tablas **estudiantes** y **cursos** preparadas.

Importante: este script borra las tablas si existen. Usarlo solo en entorno de práctica.

```
DROP TABLE cursos PURGE;
DROP TABLE estudiantes PURGE;
```

```
CREATE TABLE estudiantes (
  id_estudiante NUMBER PRIMARY KEY,
  nombre        VARCHAR2(100) NOT NULL,
  email         VARCHAR2(100) UNIQUE,
  edad          NUMBER CHECK (edad >= 16),
  ciudad        VARCHAR2(50),
  fecha_alta    DATE DEFAULT SYSDATE
);
```

```
CREATE TABLE cursos (
  id_curso      NUMBER PRIMARY KEY,
  nombre_curso  VARCHAR2(100) NOT NULL,
  categoria     VARCHAR2(50),
  horas         NUMBER CHECK (horas > 0),
  precio        NUMBER(8,2) CHECK (precio >= 0),
  activo        CHAR(1) CHECK (activo IN ('S', 'N'))
);
```

```
INSERT INTO estudiantes (id_estudiante, nombre, email, edad, ciudad, fecha
_alta)
VALUES (1, 'Ana López', 'ana.lopez@email.com', 22, 'Madrid', DATE '2026-01
-10');
```

```
INSERT INTO estudiantes (id_estudiante, nombre, email, edad, ciudad, fecha
_alta)
VALUES (2, 'Luis García', 'luis.garcia@email.com', 28, 'Valencia', DATE '2
```

```
026-01-12');
```

```
INSERT INTO estudiantes (id_estudiante, nombre, email, edad, ciudad, fecha_alta)
VALUES (3, 'María Pérez', 'maria.perez@email.com', 19, 'Madrid', DATE '2026-02-01');
```

```
INSERT INTO estudiantes (id_estudiante, nombre, email, edad, ciudad, fecha_alta)
VALUES (4, 'Carlos Ruiz', 'carlos.ruiz@email.com', 35, 'Sevilla', DATE '2026-02-15');
```

```
INSERT INTO estudiantes (id_estudiante, nombre, email, edad, ciudad, fecha_alta)
VALUES (5, 'Lucía Martín', 'lucia.martin@email.com', 24, 'Bilbao', DATE '2026-03-03');
```

```
INSERT INTO estudiantes (id_estudiante, nombre, email, edad, ciudad, fecha_alta)
VALUES (6, 'Andrés Molina', 'andres.molina@email.com', 31, 'Madrid', DATE '2026-03-20');
```

```
INSERT INTO cursos (id_curso, nombre_curso, categoria, horas, precio, activo)
VALUES (1, 'Introducción a SQL', 'Base de datos', 20, 120, 'S');
```

```
INSERT INTO cursos (id_curso, nombre_curso, categoria, horas, precio, activo)
VALUES (2, 'Oracle PL/SQL básico', 'Programación', 30, 180, 'S');
```

```
INSERT INTO cursos (id_curso, nombre_curso, categoria, horas, precio, activo)
VALUES (3, 'Administración Oracle inicial', 'Administración', 40, 250, 'S');
```

```
INSERT INTO cursos (id_curso, nombre_curso, categoria, horas, precio, activo)
VALUES (4, 'Modelado de datos', 'Base de datos', 15, 90, 'N');
```

```
INSERT INTO cursos (id_curso, nombre_curso, categoria, horas, precio, activo)
VALUES (5, 'Consultas SQL avanzadas', 'Base de datos', 25, 160, 'S');
```

```
COMMIT;
```

Comprobación rápida:

```
SELECT * FROM estudiantes;  
SELECT * FROM cursos;
```

1. Operadores lógicos y filtros específicos en Oracle SQL Developer

1.1. Operadores lógicos: **AND**, **OR**, **NOT**

Los operadores lógicos permiten combinar condiciones.

AND Se usa cuando deben cumplirse varias condiciones a la vez.

```
SELECT *  
FROM estudiantes  
WHERE ciudad = 'Madrid'  
AND edad > 25;
```

Significa:

Estudiantes de Madrid y mayores de 25 años.

OR Se usa cuando puede cumplirse una condición u otra.

```
SELECT *  
FROM estudiantes  
WHERE ciudad = 'Madrid'  
OR ciudad = 'Valencia';
```

Significa:

Estudiantes de Madrid o de Valencia.

NOT Niega una condición.

```
SELECT *  
FROM cursos  
WHERE NOT categoria = 'Base de datos';
```

Significa:

Cursos cuya categoría no sea Base de datos.

1.2. Uso de paréntesis en condiciones

Quando se mezclan **AND** y **OR**, conviene usar paréntesis para indicar claramente qué condiciones se evalúan juntas.

- Consulta con paréntesis:

```
SELECT *  
FROM estudiantes  
WHERE ciudad = 'Madrid'  
      AND (edad > 25 OR nombre LIKE 'A%');
```

Significa:

Estudiantes de Madrid que además cumplen una de estas dos condiciones:

- tienen más de 25 años;
- o su nombre empieza por A.

Otro ejemplo:

```
SELECT *  
FROM cursos  
WHERE activo = 'S'  
      AND (categoria = 'Base de datos' OR categoria = 'Programación');
```

Significa:

Cursos activos que pertenecen a Base de datos o Programación.

1.3. Búsquedas con **LIKE**

LIKE permite buscar patrones en textos.

Patrón	Significado
'A%'	empieza por A
'%a'	termina en a
'%SQL%'	contiene SQL
'_na%'	segunda y tercera letra son n y a

Ejemplos:

```
SELECT *  
FROM estudiantes  
WHERE nombre LIKE 'A%';
```

```
SELECT *  
FROM cursos
```

```
WHERE nombre_curso LIKE '%SQL%';
```

```
SELECT *  
FROM estudiantes  
WHERE email LIKE '%email.com';
```

Para evitar problemas con mayúsculas y minúsculas se puede usar `UPPER` o `LOWER` :

```
SELECT *  
FROM cursos  
WHERE UPPER(nombre_curso) LIKE '%SQL%';
```

```
SELECT *  
FROM estudiantes  
WHERE LOWER(nombre) LIKE '%a%';
```

1.4. Filtro con `IN`

`IN` permite comparar contra una lista de valores.

```
SELECT *  
FROM estudiantes  
WHERE ciudad IN ('Madrid', 'Valencia', 'Bilbao');
```

Esta consulta devuelve estudiantes cuya ciudad esté dentro de la lista indicada.

Para negar la lista:

```
SELECT *  
FROM estudiantes  
WHERE ciudad NOT IN ('Madrid', 'Valencia');
```

Ejemplo con cursos:

```
SELECT *  
FROM cursos  
WHERE categoria IN ('Base de datos', 'Programación');
```

1.5. Filtro con `BETWEEN`

`BETWEEN` permite filtrar rangos.

Ejemplo con números:

```
SELECT *  
FROM estudiantes
```

```
WHERE edad BETWEEN 20 AND 30;
```

Incluye los extremos. Es decir, incluye edad 20 y edad 30.

Ejemplo con precios:

```
SELECT *  
FROM cursos  
WHERE precio BETWEEN 100 AND 200;
```

Ejemplo con fechas:

```
SELECT *  
FROM estudiantes  
WHERE fecha_alta BETWEEN DATE '2026-01-01' AND DATE '2026-02-28';
```

2. Práctica

2.1. Comprobar los datos disponibles

```
SELECT *  
FROM estudiantes;
```

```
SELECT *  
FROM cursos;
```

Preguntas:

- ¿Qué columnas permiten buscar por texto?
- ¿Qué columnas permiten trabajar con rangos?
- ¿Qué columnas permiten listas de valores?
- ¿Qué consultas podrían necesitar paréntesis?

2.2. Consultas con operadores lógicos

Consulta 1

Mostrar estudiantes de Madrid mayores de 25 años.

```
SELECT *  
FROM estudiantes  
WHERE ciudad = 'Madrid'  
AND edad > 25;
```

Consulta 2

Mostrar estudiantes de Madrid o Valencia.

```
SELECT *  
FROM estudiantes  
WHERE ciudad = 'Madrid'  
       OR ciudad = 'Valencia';
```

Consulta 3

Mostrar cursos que no sean de la categoría `Base de datos`.

```
SELECT *  
FROM cursos  
WHERE NOT categoria = 'Base de datos';
```

2.3. Consultas con paréntesis

Consulta 4

Mostrar estudiantes de Madrid que tengan más de 25 años o cuyo nombre empiece por A.

```
SELECT *  
FROM estudiantes  
WHERE ciudad = 'Madrid'  
       AND (edad > 25 OR nombre LIKE 'A%');
```

Consulta 5

Mostrar cursos activos que sean de `Base de datos` o `Programación`.

```
SELECT *  
FROM cursos  
WHERE activo = 'S'  
       AND (categoria = 'Base de datos' OR categoria = 'Programación');
```

Consulta 6

Comparar el resultado de estas dos consultas y explicar la diferencia.

```
SELECT *  
FROM cursos  
WHERE activo = 'S'  
       AND categoria = 'Base de datos'  
       OR categoria = 'Programación';
```

```
SELECT *  
FROM cursos
```



```
WHERE activo = 'S'
AND (categoria = 'Base de datos' OR categoria = 'Programación');
```

2.4. Consultas con LIKE

Consulta 7

Mostrar estudiantes cuyo nombre empiece por A .

```
SELECT *
FROM estudiantes
WHERE nombre LIKE 'A%';
```

Consulta 8

Mostrar estudiantes cuyo nombre contenga la letra a , sin depender de mayúsculas o minúsculas.

```
SELECT *
FROM estudiantes
WHERE LOWER(nombre) LIKE '%a%';
```

Consulta 9

Mostrar cursos cuyo nombre contenga SQL .

```
SELECT *
FROM cursos
WHERE UPPER(nombre_curso) LIKE '%SQL%';
```

Consulta 10

Mostrar estudiantes cuyo correo pertenezca al dominio email.com .

```
SELECT *
FROM estudiantes
WHERE email LIKE '%email.com';
```

2.5. Consultas con IN

Consulta 11

Mostrar estudiantes de Madrid, Valencia o Bilbao.

```
SELECT *
FROM estudiantes
WHERE ciudad IN ('Madrid', 'Valencia', 'Bilbao');
```

Consulta 12

Mostrar estudiantes que no sean de Madrid ni Valencia.

```
SELECT *
FROM estudiantes
WHERE ciudad NOT IN ('Madrid', 'Valencia');
```

Consulta 13

Mostrar cursos de las categorías `Base de datos` o `Programación`.

```
SELECT *
FROM cursos
WHERE categoria IN ('Base de datos', 'Programación');
```

2.6. Consultas con `BETWEEN`

Consulta 14

Mostrar estudiantes entre 20 y 30 años.

```
SELECT *
FROM estudiantes
WHERE edad BETWEEN 20 AND 30;
```

Consulta 15

Mostrar cursos con precio entre 100 y 200 euros.

```
SELECT *
FROM cursos
WHERE precio BETWEEN 100 AND 200;
```

Consulta 16

Mostrar estudiantes dados de alta entre enero y febrero de 2026.

```
SELECT *
FROM estudiantes
WHERE fecha_alta BETWEEN DATE '2026-01-01' AND DATE '2026-02-28';
```

3. Ejercicios propuestos

- Mostrar estudiantes de Madrid que tengan más de 25 años o cuyo nombre empiece por `A`. Usar paréntesis.
- Mostrar cursos activos que sean de `Base de datos` o `Programación`. Usar paréntesis.

3. Mostrar estudiantes que no sean de Madrid ni Valencia usando `NOT IN`.
4. Mostrar cursos cuyo nombre contenga `Oracle` o `SQL`.
5. Mostrar estudiantes cuyo email contenga `garcia`.
6. Mostrar cursos activos con precio entre 100 y 200.
7. Mostrar estudiantes de Madrid o Bilbao cuya edad esté entre 20 y 30.
8. Mostrar cursos que no sean de `Base de datos` y que tengan precio entre 100 y 250.
9. Comparar una consulta con paréntesis y otra sin paréntesis donde se mezclen `AND` y `OR`.
10. Crear una consulta propia que combine `AND`, `IN` y `BETWEEN`.

Sesión 2 : Funciones SQL, cálculos y agrupaciones en Oracle SQL Developer

1.1. Función de fila

Una función de fila trabaja sobre cada registro individualmente.

Ejemplo:

```
SELECT nombre,  
       UPPER(nombre) AS nombre_mayusculas  
FROM estudiantes;
```

La función `UPPER` se aplica a cada fila de la tabla `ESTUDIANTES`.

1.2. Funciones de texto

Las funciones de texto permiten transformar o analizar cadenas de caracteres.

Función	Uso
<code>UPPER</code>	Convierte texto a mayúsculas
<code>LOWER</code>	Convierte texto a minúsculas
<code>INITCAP</code>	Pone iniciales en mayúscula
<code>LENGTH</code>	Cuenta caracteres
<code>SUBSTR</code>	Extrae parte de un texto
<code>INSTR</code>	Busca la posición de un texto dentro de otro
<code>TRIM</code>	Quita espacios al inicio y al final
<code>CONCAT</code>	Une dos textos

Ejemplos:

```
SELECT nombre,  
       UPPER(nombre) AS nombre_mayusculas,  
       LOWER(nombre) AS nombre_minusculas  
FROM estudiantes;
```

```
SELECT nombre,
       LENGTH(nombre) AS longitud_nombre
FROM estudiantes;
```

```
SELECT nombre,
       SUBSTR(nombre, 1, 3) AS tres_primeras_letras
FROM estudiantes;
```

```
SELECT email,
       INSTR(email, '@') AS posicion_arroba
FROM estudiantes;
```

1.3. Funciones numéricas

Las funciones numéricas permiten redondear, truncar o calcular valores.

Función	Uso
ROUND	Redondea un número
TRUNC	Trunca un número
MOD	Devuelve el resto de una división
ABS	Devuelve el valor absoluto

Ejemplos:

```
SELECT nombre_curso,
       precio,
       ROUND(precio * 1.21, 2) AS precio_con_iva
FROM cursos;
```

```
SELECT nombre_curso,
       precio,
       TRUNC(precio * 1.21, 1) AS precio_truncado
FROM cursos;
```

```
SELECT id_estudiante,
       MOD(id_estudiante, 2) AS resto_division_entre_2
FROM estudiantes;
```

1.4. Funciones de fecha

Oracle permite trabajar con fechas usando funciones específicas y operaciones aritméticas.

Función / operación	Uso
SYSDATE	Fecha y hora actual del servidor

Función / operación	Uso
fecha + número	Suma días a una fecha
fecha - número	Resta días a una fecha
MONTHS_BETWEEN	Calcula meses entre dos fechas
ADD_MONTHS	Suma meses a una fecha
EXTRACT	Extrae año, mes o día

Ejemplos:

```
SELECT SYSDATE AS fecha_actual
FROM dual;
```

```
SELECT nombre,
       fecha_alta,
       fecha_alta + 30 AS fecha_mas_30_dias
FROM estudiantes;
```

```
SELECT nombre,
       fecha_alta,
       ROUND(MONTHS_BETWEEN(SYSDATE, fecha_alta), 1) AS meses_desde_alta
FROM estudiantes;
```

```
SELECT nombre,
       fecha_alta,
       EXTRACT(YEAR FROM fecha_alta) AS anio_alta
FROM estudiantes;
```

2. Práctica

2.1. Transformar nombres de estudiantes

```
SELECT nombre,
       UPPER(nombre) AS mayusculas,
       LOWER(nombre) AS minusculas,
       INITCAP(nombre) AS formato_nombre
FROM estudiantes;
```

Pregunta :

¿Qué función usarías para hacer una búsqueda sin importar mayúsculas o minúsculas?

2.2. Analizar longitud de textos

```
SELECT nombre,
       LENGTH(nombre) AS caracteres_nombre,
       email,
       LENGTH(email) AS caracteres_email
FROM estudiantes;
```

2.3. Extraer partes del correo electrónico

```
SELECT email,
       SUBSTR(email, 1, 5) AS primeros_5_caracteres,
       INSTR(email, '@') AS posicion_arroba
FROM estudiantes;
```

2.4. Calcular precios con IVA

```
SELECT nombre_curso,
       precio,
       ROUND(precio * 1.21, 2) AS precio_con_iva
FROM cursos;
```

2.5. Calcular meses desde el alta

```
SELECT nombre,
       fecha_alta,
       ROUND(MONTHS_BETWEEN(SYSDATE, fecha_alta), 1) AS meses_desde_alta
FROM estudiantes;
```

3. funciones de agregación

3.1. Qué es una función de agregación ?

Una función de agregación resume varios registros y devuelve un único resultado o un resultado por grupo.

Funciones principales:

Función	Uso
COUNT	Cuenta registros
SUM	Suma valores
AVG	Calcula promedio
MIN	Obtiene el valor mínimo
MAX	Obtiene el valor máximo

Ejemplo:

```
SELECT COUNT(*) AS total_estudiantes
FROM estudiantes;
```

3.2. Diferencia entre `COUNT(*)` y `COUNT(columna)`

`COUNT(*)` cuenta todas las filas.

```
SELECT COUNT(*) AS total_filas
FROM estudiantes;
```

`COUNT(email)` cuenta solo las filas donde `email` no sea `NULL`.

```
SELECT COUNT(email) AS estudiantes_con_email
FROM estudiantes;
```

4. Práctica agregaciones y grupos

4.1. Contar estudiantes

```
SELECT COUNT(*) AS total_estudiantes
FROM estudiantes;
```

4.2. Calcular edad mínima, máxima y media

```
SELECT MIN(edad) AS edad_minima,
       MAX(edad) AS edad_maxima,
       ROUND(AVG(edad), 2) AS edad_media
FROM estudiantes;
```

4.3. Contar estudiantes por ciudad

```
SELECT ciudad,
       COUNT(*) AS total_estudiantes
FROM estudiantes
GROUP BY ciudad;
```

4.4. Contar cursos por categoría

```
SELECT categoria,
       COUNT(*) AS total_cursos
```

```
FROM cursos
GROUP BY categoria;
```

4.5. Calcular precio medio por categoría

```
SELECT categoria,
       ROUND(AVG(precio), 2) AS precio_medio
FROM cursos
GROUP BY categoria;
```

4.6. Calcular total de horas por categoría

```
SELECT categoria,
       SUM(horas) AS total_horas
FROM cursos
GROUP BY categoria;
```

4.7. Categorías con precio medio superior a 150

```
SELECT categoria,
       ROUND(AVG(precio), 2) AS precio_medio
FROM cursos
GROUP BY categoria
HAVING AVG(precio) > 150;
```

4.8. Ciudades con más de un estudiante

```
SELECT ciudad,
       COUNT(*) AS total_estudiantes
FROM estudiantes
GROUP BY ciudad
HAVING COUNT(*) > 1;
```

5. Ejercicios propuestos

1. Contar cuántos estudiantes hay en total.
2. Calcular la edad mínima, máxima y media de los estudiantes.
3. Contar cuántos estudiantes hay por ciudad.
4. Contar cuántos cursos hay por categoría.
5. Calcular el precio medio de los cursos.

6. Calcular el precio medio por categoría.
7. Calcular el total de horas por categoría.
8. Mostrar solo las ciudades que tengan más de un estudiante.
9. Mostrar solo las categorías cuyo precio medio sea mayor que 150.
10. Crear una consulta propia que use una función de texto y una función de agregación.